

쇼핑몰 상품 크롤러 — 처음 보는 사람을 위한 설명서

코딩을 몰라도 이 프로그램이 뭘 하는지, 어떻게 실행하는지, 결과물이 뭔지 이해할 수 있도록 쓴 안내서입니다. 짧은 요약은 [README.md](#) 를 참고하세요.

1. 이게 뭔가요?

"크롤러"(또는 "스크래퍼")란 사람이 웹사이트를 하나하나 열어보면서 정보를 복사-붙여넣기 하는 작업을, 프로그램이 대신 자동으로 해주는 도구입니다. 이 프로젝트는 [scrapeme.live](#) 라는 곳에서 **크롤링 연습용으로 공개해 둔 가상의 포켓몬 쇼핑몰**에서 상품명과 가격을 자동으로 긁어와 엑셀 파일로 정리해주는 프로그램입니다.

실제 외주에서는 이 구조를 그대로 활용해서, 예를 들어 "경쟁사 쇼핑몰의 상품 가격을 자동으로 모아서 엑셀로 보내주는" 가격 비교 서비스 같은 걸 만들 수 있습니다.

이 프로그램은 **크롤링을 시작하기 전에 그 사이트가 크롤링을 허용하는지 자동으로 확인**합니다 (자세한 내용은 8번 참고). 아무 사이트나 무작정 긁지 않고, 정해진 규칙을 지키도록 만들어졌다는 점이 이 크롤러의 핵심 특징입니다.

2. 파일이 뭐가 있나요?

```
web-crawler-books/
├─ scraper.py      ← 크롤링 + 저장 로직 (페이지 기반, 실행 시작점)
├─ ai_extractor.py ← 셀렉터 없는 사이트를 AI(Gemini)로 대신 읽어주는 보조 기능
├─ robots_check.py ← 크롤링 전 robots.txt로 허용 여부를 확인하는 안전장치
├─ http_utils.py  ← 네트워크 요청 실패 시 자동 재시도해주는 공용 기능
├─ sqlite_store.py ← 가격 변동 이력을 계속 누적해서 저장하는 기능
├─ .env.example   ← AI 기능에 필요한 API 키 설정 예시
├─ README.md      ← 프로젝트 요약
├─ 설명서.md      ← 지금 보고 있는 이 문서
└─ .gitignore     ← 실행 결과 파일/비밀 키 파일은 저장소에 올리지 않도록 제외 설정
```

실행하면 [output_products.csv](#), [output_products.xlsx](#), [output_products.json](#), [price_history.db](#) 라는 결과 파일이 새로 생기는데, 이 파일들은 저장소에는 포함되어 있지 않고 실행할 때마다 새로 만들어집니다(단, [price_history.db](#) 는 "뒹어쓰기"가 아니라 "계속 쌓이는" 파일입니다 — 자세한 내용은 11번 참고).

3. 어떻게 실행하나요?

이 프로그램은 파이썬(Python)으로 만들어져 있어서, 더블클릭으로 바로 열리는 `index.html` 과 달리 **컴퓨터에 파이썬이 설치되어 있어야** 실행할 수 있습니다.

1. 파이썬이 설치돼 있는지 확인 (없으면 python.org에서 설치)
2. 아래 명령어로 필요한 부품(라이브러리) 설치 (한 번만 하면 됨)

```
pip install requests beautifulsoup4 xlswriter
```
3. 이 폴더 위치에서 아래 명령어 실행

```
python scraper.py
```
4. 실행이 끝나면 같은 폴더에 `output_products.csv` , `output_products.xlsx` 파일이 생성됨 — 엑셀로 열어서 바로 확인 가능

몇 페이지까지 수집할지 조절하고 싶으면 이렇게 실행합니다.

```
python scraper.py --pages 5
```

4. 실행하면 내부에서 무슨 일이 일어나나요? (쉬운 설명)

사람이 직접 한다면 이런 순서로 하게 될 작업을, 프로그램이 몇 초 만에 대신 해줍니다.

1. 1페이지부터 순서대로, 지정한 페이지 수만큼 처리
2. **각 페이지를 가져오기 전에, 이 사이트가 크롤링을 허용하는지 먼저 확인** (허용 안 하면 그 자리에서 멈춤)
3. 페이지 화면을 인터넷에서 가져옴 — **이때 인터넷이 잠깐 끊기는 등 일시적인 문제가 생기면, 포기하지 않고 몇 초 기다렸다가 자동으로 다시 시도함** (최대 3번까지)
4. 그 화면 안에서 상품 하나하나의 "이름"과 "가격"이 적힌 부분만 정확히 찾아서 뽑아냄 (할인 중인 상품은 할인 후 실제 판매가를 가져옴)
5. **그 페이지에서 찾은 정보를 CSV 파일에 바로 이어서 저장** — 모든 페이지를 다 돌 때까지 기다리지 않고, 페이지 하나가 끝날 때마다 즉시 기록함
6. 다음 페이지로 넘어가기 전, 서버에 부담을 주지 않도록 0.5초 대기
7. 지정한 페이지를 다 처리하면(또는 중간에 중단되면), 그때까지 CSV에 저장된 내용을 바탕으로 Excel 파일(색깔·굵은 글씨 등 서식이 적용된 표)을 만들

참고: 원래는 다른 연습용 사이트(가짜 서점, 가짜 전자제품 쇼핑몰)를 대상으로 만들었는데, 그 사이트의 robots.txt를 직접 확인해보니 해당

경로가 크롤링 금지로 설정되어 있는 걸 발견해서, 실제로 크롤링이 허용된 `scrapeme.live` 로 다시 교체했습니다.

5. 결과물(엑셀 파일)은 어떻게 생겼나요?

상품명	가격(GBP)	페이지
Bulbasaur	63.00	1
Machoke	27.00	5

- 맨 위 줄(제목 줄)은 진한 배경색으로 강조되어 있고, 스크롤해도 항상 보이도록 고정되어 있습니다
- 가격 옆은 통화 기호(£)가 자동으로 붙는 서식이 적용돼 있습니다

6. 실행 옵션 바꾸는 법

명령어 뒤에 옵션을 붙여서 동작을 조절할 수 있습니다.

옵션	뜻	예시
<code>--pages</code>	몇 페이지까지 수집할지 (기본 3페이지)	<code>python scraper.py --pages 5</code>
<code>--csv</code>	CSV 파일 저장 경로/이름 지정	<code>python scraper.py --csv 결과.csv</code>
<code>--excel</code>	Excel 파일 저장 경로/이름 지정	<code>python scraper.py --excel 결과.xlsx</code>
<code>--json</code>	JSON 파일 저장 경로/이름 지정	<code>python scraper.py --json 결과.json</code>
<code>--db</code>	가격 이력을 쌓을 SQLite 파일 경로 지정	<code>python scraper.py --db 이력.db</code>
<code>--history</code>	크롤링 대신, 키워드가 포함된 상품의 과거 가격을 조회	<code>python scraper.py --history "Bulbasaur"</code>

7. 다른 사이트에도 이 프로그램을 쓸 수 있나요?

네, 하지만 사이트마다 화면 구조가 달라서 코드를 그 사이트에 맞게 고쳐야 합니다. 지금 코드는 이 특정 쇼핑몰의 화면 구조(어느 위치에 상품명이고, 어느 위치에 가격이 있는지)에 맞춰져 있습니다. 실제 클라이언트의 사이트에 쓰려면 그 사이트 구조를 분석해서 같은 방식으로 다시 맞춰야 합니다.

8. 크롤링 전 자동으로 허락을 구하는 장치 (robots.txt 확인)

⚠️ **크롤링하기 전에 반드시 확인할 것:** 아무 사이트나 마음대로 크롤링하면 안 됩니다. 대부분의 웹사이트는 `robots.txt` 라는 파일에 "여기는 크롤링해도 되고, 여기는 안 된다"는 규칙을 적어둡니다 (예: `그사이트주소.com/robots.txt` 로 들어가면 볼 수 있음).

이 프로그램은 이 규칙을 **사람이 매번 확인하지 않아도 코드가 자동으로 확인**하도록 만들어져 있습니다 (`robots_check.py`). 페이지를 가져오기 직전에 항상 이 확인을 거치고, 만약 그 사이트가 해당 경로의 크롤링을 금지하고 있다면 프로그램이 그 자리에서 스스로 멈추고 이유를 알려줍니다.

실제로 개발 중 이 기능 덕분에, 원래 쓰려고 했던 다른 연습용 사이트 하나가 실제로는 해당 경로 크롤링을 금지하고 있다는 걸 발견했고, 그래서 진짜로 허용된 사이트로 교체했습니다. 로그인이 필요한 개인정보나 저작권이 있는 콘텐츠는 robots.txt에 나타나지 않는 경우도 있으니, 그런 부분은 robots.txt 확인과 별개로 항상 수집 대상에서 제외해야 합니다.

9. AI가 대신 읽어주는 기능 (셀렉터가 없는 사이트용)

지금까지 설명한 방식은 "이 사이트의 상품명은 여기, 가격은 여기"라고 사람이 미리 정확한 위치를 알려줘야만 동작합니다. 그런데 **아직 위치를 알려주지 않은, 처음 보는 사이트**를 급하게 시도해봐야 할 때도 있습니다. 이럴 때 쓰는 게 `--ai-url` 기능입니다 (이 기능도 8번과 동일하게 요청 전 robots.txt를 자동으로 확인합니다).

동작 방식은 이렇습니다.

1. 지정한 페이지의 내용을 가져와서 사람이 읽는 것과 비슷한 순수 텍스트로 정리
2. 그 텍스트를 구글의 AI(Gemini)에게 보여주면서 "여기서 상품명과 가격을 찾아서 정리해줘"라고 요청
3. AI가 정리해서 돌려준 결과를 기존과 똑같은 엑셀/CSV 형식으로 저장

사용하려면 먼저 무료 AI 키를 준비해야 합니다.

1. aistudio.google.com에서 구글 계정으로 로그인
2. "API 키 만들기"로 무료 키 발급 (신용카드 등록 불필요)
3. `.env.example` 파일을 복사해서 `.env` 라는 이름으로 저장하고, 그 안에 발급받은 키를 붙여넣기 (이 파일은 나만 보는 파일이라 GitHub에는 절대 올라가지 않습니다)
4. 아래처럼 실행

```
python scraper.py --ai-url "https://대상사이트.com/상품목록"
```

⚠️ **이 방식의 한계:** AI는 화면에 "글자로 보이는 정보"만 읽을 수 있습니다. 별점이 숫자 텍스트가 아니라 그림(아이콘)으로만 표시된 사이트는 그 정보를 읽어내지 못합니다. 반면 처음부터 위치를 정확히 알려준 기존 방식(`scrape_products`)은 더 정확합니다. 즉, **AI 방식은 "빠르게 아무 사이트나 시도해볼 때", 기존 방식은 "특정 사이트를 정확하고 빠르게 반복 수집할 때"** 쓰는 것으로 구분하면 됩니다.

10. 중간에 문제가 생겨도 데이터가 안전한 이유

처음 만들었을 때는 "다 모아야 저장" 방식이라, 수집 도중 인터넷이 끊기면 그동안 모은 데이터가 전부 사라지는 문제가 있었습니다. 지금은 아래 두 가지 안전장치가 추가되어 있습니다.

1. **자동 재시도**: 요청 하나가 실패하면 바로 포기하지 않고, 2초 → 4초로 점점 길게 기다리면서 최대 3번까지 다시 시도합니다. 그래도 안 되면 그 페이지만 건너뛰고 나머지는 계속 진행합니다 (프로그램 전체가 죽지 않음)
2. **페이지 단위 즉시 저장**: 페이지 하나를 다 읽을 때마다 결과를 바로 CSV 파일에 이어서 기록합니다. 그래서 5페이지 중 4페이지째에서 중단되더라도, 앞의 3페이지 결과는 이미 파일에 안전하게 남아있습니다. 프로그램을 다시 실행하면 이전 파일은 새로 덮어쓰고 시작하므로, 예전 데이터와 새 데이터가 뒤섞이는 일도 없습니다

참고: 개발 중 재시도 메시지 안에 있던 특수문자(—) 하나가 Windows 콘솔에서 깨지면서, "재시도하다가 오히려 프로그램이 죽는" 문제가 실제로 있었습니다. 지금은 수정되어 있습니다.

11. 결과를 다른 형식으로도 저장하고, 가격 변동을 추적하는 기능

파일 두 가지(CSV, Excel)만으로는 아쉬운 상황이 있어서 두 가지를 추가했습니다.

JSON 저장: 이 크롤러의 결과를 다른 프로그램이나 웹사이트에서 바로 가져다 쓰고 싶을 때 쓰는 형식입니다. 개발자에게 데이터를 넘겨줄 때 Excel보다 JSON이 훨씬 다루기 편합니다.

SQLite로 가격 이력 누적: CSV/Excel/JSON은 실행할 때마다 이전 내용을 지우고 최신 결과로 덮어씁니다. 반면 `price_history.db` 는 **지우지 않고 계속 쌓입니다**. 그래서 이 크롤러를 매일 한 번씩 실행하도록 예약해두면, "이 상품이 지난주엔 얼마였는데 오늘은 얼마로 바뀌었는지" 같은 **가격 변동 추적**이 가능해집니다. 이걸 실제로 "경쟁사 가격 모니터링" 같은 정기 서비스를 팔 때 핵심이 되는 기능입니다.

조회는 이렇게 합니다.

```
python scraper.py --history "Bulbasaur"
```

그러면 그 이름이 포함된 상품의 가격을 시간 순서대로 보여줍니다. 예:

```
2026-08-22T08:26:51+00:00 | Bulbasaur | GBP 63.0
2026-08-22T08:27:25+00:00 | Bulbasaur | GBP 63.0
```

참고: 구글 시트에 결과가 자동으로 채워지는 기능도 만들 수 있지만, 구글 클라우드에 별도 "서비스 계정"을 만들고 시트 편집 권한을 공유해줘야 하는 등 설정이 꽤 복잡해서, 지금은 만들지 않고 "요청 시 가능"한 옵션으로만 남겨뒀습니다.

12. 현재 알려진 한계 (실제 납품 전 보완 필요)

- **대량 수집에는 느림:** 페이지 수가 많아지면 시간이 오래 걸릴 수 있어, 그런 경우 더 빠른 방식(Scrapy 등)으로 다시 만드는 게 좋습니다
- **사이트 구조 자체가 바뀌는 경우는 대응 못 함:** 재시도는 "일시적인 네트워크 문제"에만 도움이 되고, 사이트 디자인이 통째로 바뀌어 셀렉터가 안 맞게 되는 경우는 코드를 다시 고쳐야 합니다

13. 자주 헛갈릴 수 있는 용어 정리

용어	쉬운 설명
크롤링/스크래핑	웹사이트의 정보를 자동으로 읽어와 모으는 작업
Python(파이썬)	이 프로그램을 만든 프로그래밍 언어. 실행하려면 컴퓨터에 설치돼 있어야 함
라이브러리	남이 미리 만들어둔 기능 모음. <code>pip install</code> 로 설치해서 가져다 씀
CSV	표 데이터를 저장하는 가장 단순한 파일 형식 (메모장으로도 열림)
robots.txt	그 사이트에서 크롤링해도 되는 범위를 적어둔 규칙 파일
API 키	그 서비스(여기서는 구글 AI)를 프로그램에서 사용할 수 있게 해주는 비밀번호 같은 값. 절대 다른 사람과 공유하면 안 됨
.env 파일	API 키처럼 남에게 보여주면 안 되는 값을 따로 모아두는 파일. GitHub에는 올라가지 않도록 설정돼 있음
프롬프트 (Prompt)	AI에게 "무엇을 해달라"고 요청하는 지시문
JSON	프로그램끼리 데이터를 주고받을 때 자주 쓰는 표준 형식. 사람이 읽기엔 CSV/Excel보다 불편하지만 개발자에게는 더 편함
데이터베이스 (DB)	많은 양의 데이터를 체계적으로 저장하고 검색할 수 있게 해주는 저장소. 엑셀 파일보다 대량의 데이터를 다루거나, 조건에 맞는 데이터만 빠르게 찾을 때 유리함
SQLite	별도 설치나 서버 없이 파일 하나로 동작하는 가벼운 데이터베이스. 이 프로젝트의 가격 이력 저장에 사용됨